logic.py

```python
import numpy as np
import matplotlib.pyplot as plt


class FuzzyMembershipFunction:

    @staticmethod
    def triangular(x, a, b, c):
        """Triangular membership function"""

        if x <= a or x >= c:
            return 0

        elif a < x <= b:
            return (x - a) / (b - a)

        else:
            return (c - x) / (c - b)

    @staticmethod
    def trapezoidal(x, a, b, c, d):
        """Trapezoidal membership function"""

        if x <= a or x >= d:
            return 0

        elif a < x <= b:
            return (x - a) / (b - a)

        elif b <= x <= c:
            return 1

        else:
            return (d - x) / (d - c)


class FuzzyTemperatureController:

    def __init__(self):
        self.temp_range = (0, 35)
        self.humidity_range = (0, 100)
        self.speed_range = (0, 100)

    def fuzzify_temperature(self, temp):
        """Convert crisp temperature to fuzzy values"""

        cold = FuzzyMembershipFunction.triangular(
            temp, 0, 5, 15
```

```
        )

        warm = FuzzyMembershipFunction.triangular(
            temp, 10, 17.5, 25
        )

        hot = FuzzyMembershipFunction.triangular(
            temp, 20, 27.5, 35
        )

        return {
            'cold': cold,
            'warm': warm,
            'hot': hot
        }

    def fuzzify_humidity(self, humidity):
        """Convert crisp humidity to fuzzy values"""

        low = FuzzyMembershipFunction.triangular(
            humidity, 0, 20, 40
        )

        medium = FuzzyMembershipFunction.triangular(
            humidity, 30, 50, 70
        )

        high = FuzzyMembershipFunction.triangular(
            humidity, 60, 80, 100
        )

        return {
            'low': low,
            'medium': medium,
            'high': high
        }

    def fuzzify_fan_speed(self, speed):
        """Membership functions for fan speed"""

        off = FuzzyMembershipFunction.triangular(
            speed, 0, 0, 20
        )

        low = FuzzyMembershipFunction.triangular(
            speed, 10, 30, 50
        )

        medium = FuzzyMembershipFunction.triangular(
            speed, 40, 60, 80
```

```
        )

        high = FuzzyMembershipFunction.triangular(
            speed, 70, 100, 100
        )

        return {
            'off': off,
            'low': low,
            'medium': medium,
            'high': high
        }

    def apply_rules(self, temp_fuzzy, humidity_fuzzy):
        """Apply fuzzy rules and return output"""

        rules_fired = []

        # Rule 1: IF temp is Cold AND humidity is Low THEN speed is Off
        rule1 = min(
            temp_fuzzy['cold'],
            humidity_fuzzy['low']
        )

        rules_fired.append(('off', rule1))

        # Rule 2: IF temp is Warm AND humidity is Low THEN speed is Low
        rule2 = min(
            temp_fuzzy['warm'],
            humidity_fuzzy['low']
        )

        rules_fired.append(('low', rule2))

        # Rule 3: IF temp is Warm AND humidity is Medium THEN speed is Medium
        rule3 = min(
            temp_fuzzy['warm'],
            humidity_fuzzy['medium']
        )

        rules_fired.append(('medium', rule3))

        # Rule 4: IF temp is Hot AND humidity is Low THEN speed is Low
        rule4 = min(
            temp_fuzzy['hot'],
            humidity_fuzzy['low']
        )

        rules_fired.append(('low', rule4))

```

```
        # Rule 5: IF temp is Hot AND humidity is Medium THEN speed is Medium
        rule5 = min(
            temp_fuzzy['hot'],
            humidity_fuzzy['medium']
        )

        rules_fired.append(('medium', rule5))

        # Rule 6: IF temp is Hot AND humidity is High THEN speed is High
        rule6 = min(
            temp_fuzzy['hot'],
            humidity_fuzzy['high']
        )

        rules_fired.append(('high', rule6))

        return rules_fired

    def defuzzify(self, rules_fired):
        """Center of Mass defuzzification"""

        numerator = 0
        denominator = 0

        for speed_level, strength in rules_fired:

            if speed_level == 'off':
                center = 0

            elif speed_level == 'low':
                center = 30

            elif speed_level == 'medium':
                center = 60

            elif speed_level == 'high':
                center = 90

            numerator += center * strength
            denominator += strength

        if denominator == 0:
            return 0

        return numerator / denominator

    def control(self, temperature, humidity):
        """Main control logic"""

        # Fuzzification
```

```
        temp_fuzzy = self.fuzzify_temperature(temperature)
        humidity_fuzzy = self.fuzzify_humidity(humidity)

        # Rule Evaluation
        rules_fired = self.apply_rules(
            temp_fuzzy,
            humidity_fuzzy
        )

        # Defuzzification
        fan_speed = self.defuzzify(rules_fired)

        return (
            fan_speed,
            temp_fuzzy,
            humidity_fuzzy,
            rules_fired
        )


# ============================================================
# Create Controller
# ============================================================

print("\n" + "#" * 70)
print("FUZZY LOGIC TEMPERATURE AND FAN SPEED CONTROLLER")
print("#" * 70)

controller = FuzzyTemperatureController()


# ============================================================
# Test Cases
# ============================================================

test_cases = [
    (10, 30, "Cold room, low humidity"),
    (18, 45, "Warm room, medium humidity"),
    (28, 75, "Hot room, high humidity"),
    (5, 20, "Very cold, very dry"),
    (32, 90, "Very hot, very humid"),
]

print("\n" + "=" * 70)
print("TEST RESULTS")
print("=" * 70)

print(
    f"{'Temp(°C)':<12} "
    f"{'Humidity(%)':<15} "
```

```
    f"{'Fan Speed(%)':<15} "
    f"{'Scenario':<25}"
)

print("-" * 70)

for temp, humidity, scenario in test_cases:

    fan_speed, _, _, _ = controller.control(
        temp,
        humidity
    )

    print(
        f"{temp:<12} "
        f"{humidity:<15} "
        f"{fan_speed:<15.2f} "
        f"{scenario:<25}"
    )

print("=" * 70)


# ============================================================
# Detailed Analysis for One Case
# ============================================================

print("\n\n" + "#" * 70)
print("DETAILED ANALYSIS: Hot Room with High Humidity")
print("#" * 70)

temperature = 28
humidity = 75

fan_speed, temp_fuzzy, humidity_fuzzy, rules_fired = (
    controller.control(
        temperature,
        humidity
    )
)


# Input Conditions
print("\nInput Conditions:")
print(f"  Temperature: {temperature}°C")
print(f"  Humidity: {humidity}%")


# Fuzzification Results
print("\nFuzzification Results:")
```

```
299
300 print("\n  Temperature Fuzzy Values:")
301 print(f"    Cold: {temp_fuzzy['cold']:.4f}")
302 print(f"    Warm: {temp_fuzzy['warm']:.4f}")
303 print(f"    Hot: {temp_fuzzy['hot']:.4f}")
304
305 print("\n  Humidity Fuzzy Values:")
306 print(f"    Low: {humidity_fuzzy['low']:.4f}")
307 print(f"    Medium: {humidity_fuzzy['medium']:.4f}")
308 print(f"    High: {humidity_fuzzy['high']:.4f}")
309
310
311 # Rules Fired
312 print("\nRules Fired:")
313
314 for speed_level, strength in rules_fired:
315
316     if strength > 0:
317         print(
318             f"  {speed_level.capitalize()}: "
319             f"{strength:.4f}"
320         )
321
322
323 # Defuzzification
324 print("\nDefuzzification (Center of Mass):")
325 print(f"  Final Fan Speed: {fan_speed:.2f}%")
326
327
328 # Fan Status
329 if fan_speed < 20:
330     status = "OFF"
331
332 elif fan_speed < 50:
333     status = "LOW"
334
335 elif fan_speed < 80:
336     status = "MEDIUM"
337
338 else:
339     status = "HIGH"
340
341
342 print(f"  Status: {status}")
343
344 print("=" * 70)
```

```
##########################################################################
MEDICAL DIAGNOSIS EXPERT SYSTEM
##########################################################################

Patient Symptoms:
  fever: Present
  cough: Present
  sore_throat: Present
  body_ache: Absent
  chills: Absent
  shortness_of_breath: Absent
  chest_pain: Absent

==========================================================================
FORWARD CHAINING INFERENCE ENGINE
==========================================================================

Initial Facts:
  fever: True
  cough: True
  sore_throat: True
  body_ache: False
  chills: False
  shortness_of_breath: False
  chest_pain: False

--- Iteration 1 ---
✓ Rule fired: Diagnosis: Common Cold
  Conditions met: {'fever': True, 'cough': True, 'sore_throat': True}
✓ Rule fired: Diagnosis: Possible Respiratory Infection
  Conditions met: {'fever': True, 'cough': True}
✓ Rule fired: Recommendation: Monitor temperature closely
  Conditions met: {'fever': True}
✓ Rule fired: Recommendation: Drink warm liquids and rest
  Conditions met: {'cough': True}

--- Iteration 2 ---
No new facts derived. Inference complete.
```

Activate Windows

```
DIAGNOSIS RESULTS
======================================================================

Possible Diagnoses:
  1. Diagnosis: Common Cold
  2. Diagnosis: Possible Respiratory Infection
  3. Recommendation: Monitor temperature closely
  4. Recommendation: Drink warm liquids and rest

======================================================================


######################################################################
EXAMPLE 2: DIFFERENT PATIENT WITH DIFFERENT SYMPTOMS
######################################################################

Patient 2 Symptoms:
  fever: Present
  cough: Present
  sore_throat: Absent
  body_ache: Present
  chills: Present
  shortness_of_breath: Absent
  chest_pain: Absent

======================================================================
FORWARD CHAINING INFERENCE ENGINE
======================================================================

Initial Facts:
  fever: True
  cough: True
  sore_throat: False
  body_ache: True
  chills: True
  shortness_of_breath: False
  chest_pain: False

--- Iteration 1 ---
✓ Rule fired: Diagnosis: Flu (Influenza)
  Conditions met: {'fever': True, 'cough': True, 'body_ache': True, 'chills': True}
✓ Rule fired: Diagnosis: Possible Respiratory Infection
  Conditions met: {'fever': True, 'cough': True}
✓ Rule fired: Recommendation: Monitor temperature closely
  Conditions met: {'fever': True}

--- Iteration 2 ---
No new facts derived. Inference complete.

======================================================================
DIAGNOSIS RESULTS
======================================================================

Possible Diagnoses:
  1. Diagnosis: Flu (Influenza)
  2. Diagnosis: Possible Respiratory Infection
  3. Recommendation: Monitor temperature closely
```